Ali Abdullah Ahmad
20031246
CS 600

Q1. No. 17.8.8.

(a) Given the CNF formula $B = (x_1) \cdot (\overline{x_2} + x_3 + x_5 + \overline{x_6}) \cdot (x_1 + x_4) \cdot (x_3 + \overline{x_5})$, show the reduction of B into an equivalent input for the 3SAT problem.

Feedback

Ans.

Given:
$B = (x_1) \cdot (x_2 + x_3 + x_5 + x_6) \cdot (x_1 + x_4) \cdot (x_3 + x_5)$
Let us break B into parts and apply local replacements to convert it into 3-CNF form (each clause having exactly 3 literals):

$B_1 = (x_1)$
This is a 1-literal clause. Introduce two new variables x_7 and x_8:
$B_1 = (x_1 + \neg x_7 + \neg x_8) \cdot (x_1 + \neg x_7 + x_8) \cdot (x_1 + x_7 + \neg x_8) \cdot (x_1 + x_7 + x_8)$

$B_2 = (x_2 + x_3 + x_5 + x_6)$
This is a 4-literal clause. Introduce three new variables x_9, x_{10}, and x_{11}:
$B_2 = (x_2 + x_3 + x_9) \cdot (\neg x_9 + x_5 + x_{10}) \cdot (\neg x_{10} + x_6 + x_{11})$

$B_3 = (x_1 + x_4)$
This is a 2-literal clause. Reuse x_{11}:
$B_3 = (x_1 + x_4 + x_{11}) \cdot (x_1 + x_4 + \neg x_{11})$

$B_4 = (x_3 + x_5)$
This is a 2-literal clause. Introduce a new variable x_{12}:
$B_4 = (x_3 + x_5 + x_{12}) \cdot (x_3 + x_5 + \neg x_{12})$

Therefore, the 3SAT equivalent formula is:
B =
$(x_1 + \neg x_7 + \neg x_8) \cdot (x_1 + \neg x_7 + x_8) \cdot (x_1 + x_7 + \neg x_8) \cdot (x_1 + x_7 + x_8) \cdot$
$(x_2 + x_3 + x_9) \cdot (\neg x_9 + x_5 + x_{10}) \cdot (\neg x_{10} + x_6 + x_{11}) \cdot$
$(x_1 + x_4 + x_{11}) \cdot (x_1 + x_4 + \neg x_{11}) \cdot$
$(x_3 + x_5 + x_{12}) \cdot (x_3 + x_5 + \neg x_{12})$

Q2. No. 17.8.12

(a) Professor Amongus has just designed an algorithm that can take any graph G with n vertices and determine in $O\left(n^k\right)$ time whether G contains a clique of size k. Does Professor Amongus deserve the Turing Award for having just shown that ***P*** = ***NP***? Why or why not?

Ans.

Professor Amongus' algorithm does not prove P = NP and does not deserve the Turing Award for this claim.
The clique problem is NP-complete, and while the algorithm solves it in O(n^k) time for some k, it does not provide a polynomial-time solution for all NP problems, which is required to prove P = NP.
To show P = NP, a polynomial-time algorithm must solve **all** NP-complete problems, not just a specific case.
Additionally, without proof of the algorithm's correctness or general applicability, it cannot be considered a major contribution that would warrant the Turing Award. The P = NP problem remains unsolved in computer science.

Q3. No. 17.8.28
Define HYPER-COMMUNITY to be the problem that takes a collection of n web pages and an integer k ,and determines if there are k web pages that all contain hyperlinks to each other. Show that HYPER-COMMUNITY is *NP*-complete.
Ans.

HYPER-COMMUNITY is NP-complete:

1. HYPER-COMMUNITY is in NP: A non-deterministic machine can guess k web pages and check if they are all mutually connected by hyperlinks, which can be verified in polynomial time.
2. Reduction from Independent Set:
    - Given a graph G and integer k, construct a new graph G' where:
        - There is an edge between vertices u and v in G' if and only if there is no edge between u and v in G.
    - If there is an independent set of size k in G, the corresponding vertices will form a clique in G', as they are not adjacent in G.
3. Conclusion: Since Independent Set is NP-complete and can be reduced to HYPER-COMMUNITY in polynomial time, HYPER-COMMUNITY is NP-complete.

Q4. No. 17.8.35

Imagine that the annual university job fair is scheduled for next month and it is your job to book companies to host booths in the large Truman Auditorium during the fair. Unfortunately, at last year's job fair, a fight broke out between some people from competing companies, so the university president, Dr. Noah Drama, has issued a rule that prohibits any pair of competing companies from both being invited to this year's event. In addition, he has shown you a website that lists the competitors for every company that might be invited to this year's job fair and he has asked you to invite the maximum number of noncompeting companies as possible. Show that the decision version of the problem Dr. Drama has asked you to solve is *NP*-complete.

Ans.

The problem asks us to invite the maximum number of non-competing companies to a job fair, with the condition that no two competing companies can be invited. This problem is NP-complete.

1. The problem is in NP:
    - A non-deterministic machine can guess a set of k companies and verify in polynomial time whether no two selected companies are competitors (i.e., checking that there are no edges between them). If this condition is met, the solution is valid, so the problem is in NP.
2. Proof that the problem is NP-hard:
    - Reduction from Vertex Cover: We reduce the Vertex Cover problem, which is NP-complete, to this problem.
    - In the Vertex Cover problem, the task is to find a set of at most k vertices such that every edge in the graph is covered by at least one vertex in the set.
    - For the job fair problem, we construct the complement graph where edges represent non-competing companies. Finding the largest independent set (non-competing companies) in this graph is equivalent to finding a minimum vertex cover in the original graph.
    - Since Vertex Cover is NP-complete, this reduction shows that the job fair problem is also NP-complete.

Thus, the job fair problem is NP-complete.

Q5. No. 18.6.19

EXERCISE | 18.6.19

(a) Consider the KNAPSACK problem, but now instead of implementing the PTAS algorithm given in the book, we use a greedy approach of always picking the next item that maximizes the ratio of value over weight (as in the optimal way to solve the fractional version of the KNAPSACK problem). Show that this approach does not produce a c-approximation algorithm, for any fixed value of c.

Ans.

We show that the greedy algorithm, which picks items by highest value-to-weight ratio, does not guarantee a c-approximation for the 0/1 Knapsack problem.

Counterexample:

- Items:
  - n small items: each has value 1, tiny weight epsilon (very small), value-to-weight ratio 1/epsilon(very large).
  - 1 large item: value W, weight W, value-to-weight ratio 1.
- Knapsack capacity: W.

Greedy Behavior:

- Greedy selects small items first due to high value-to-weight ratio.
- Total value collected is much smaller than W.

Optimal Solution:

- Pick the large item (value W).

Conclusion:

- Greedy's value can be made arbitrarily smaller than optimal by choosing small enough epsilon.
- Thus, the greedy approach does not achieve a c-approximation for any fixed c

Greedy fails for any fixed c

Q6. No. 18.6.26
Suppose you are preparing an algorithm for the problem of optimally drilling the holes in an aluminum plug plate to allow it to do a spectrographic analysis of a set of galaxies. Based on your analysis of the robot drill device, you notice that the various amounts of time it takes to move between drilling holes satisfies the triangle inequality. Nevertheless, your supervisor does not want you to use the MST approximation algorithm or the Christofides approximation algorithm. Instead, your supervisor wants you to use a nearest-neighbor greedy algorithm for solving this instance of METRIC-TSP. In this greedy algorithm, one starts with city number 1 as the "current" city, and then repeatedly chooses the next city to add to the tour to be the one that is closest to the current city (then making the added city to be the "current" one). Show that your supervisor's nearest-neighbor greedy algorithm does not, in general, result in a 2-approximation algorithm for METRIC-TSP.
Ans.

The nearest-neighbor greedy algorithm does not guarantee a 2-approximation for Metric-TSP.

Idea:

- Nearest neighbor starts at city 1 and always picks the closest unvisited city.
- It can make bad early choices, leading to a much longer tour.

Counterexample Sketch:

- Create a set of cities where city 1 is closest to an isolated city.
- Visiting that isolated city first forces long travel later.
- The greedy tour can be much longer than the optimal tour, by any factor greater than 2.

Conclusion: Nearest-neighbor can perform arbitrarily badly and thus is not a 2-approximation algorithm for Metric-TSP.

Nearest-neighbor is not a 2-approximation